

实验三、分支预测

实验内容

本次实验使用 SimpleScalar 下面的分支预测模拟器 sim-bpred, 需要在 4 种预测技术及其不同的参数配置下运行测试程序, 并比较、分析结果, 加深对动态分支预测机制的理解, 并了解各种分支预测技术的优劣。

实验目的

- 了解分支预测技术的基本原理
- 比较各种分支预测技术的性能

实验方法

SimpleScalar 模拟器中分支预测的实现方法: 先进行分支方向探测, 即是否跳转? (当然, 绝对跳转指令和函数调用及返回指令不用作这一步)。接着是预测分支目标的地址: 对于函数调用返回指令, 直接在 RAS 上作相关操作; 普通分支指令则要利用 BTB 来进行地址探测, 命中则获得目标地址。然后对上述两步综合处理: 若 BTB 命中且分支预测为跳转, 返回分支目标地址; 若 BTB 缺失且分支预测为跳转, 返回 1; 只要分支预测为不跳转, 就返回 0。

针对条件分支指令的方向探测方法, 主要有 5 种, 三种静态: taken, nottaken; 三种动态: bimod, 2-level, combined。本实验选择 4 种方法开展实验: 2 种静态的分支预测方法, 以及 2 种动态的分支预测方法 (bimod 与 2-level)。

对于动态方法, 说明如下:

bimod 是最普通的, 即采用一个 2bit 宽的分支方向预测表, 按分支地址查找, 2bit 分支预测器的判断和更新与课本上的一致。这种方式只有一个参数, 就是分支预测表的长度。

2-level 要复杂一些, 它采用两级表格式, 第一级是分支历史表, 存放各组分支历史寄存器的值, 第二级是全局/局部分支模式表, (全局或局部应是由表长相对于分支历史寄存器的长决定), 它存放各分支历史模式的 2bit 预测器。在判断时用当前分支指令对应的历史寄存器值去索引二级表得到相应预测器值。

更新时，把当前分支的方向左移入历史寄存器，并对使用过的 2bit 预测器作更新。它有四个参数，前三个是：一级表长度，二级表长度，历史寄存器宽度，最后一个为异或标志。如果为 1，则将历史寄存器的值与当前分支指令地址异或，用其结果再去索引二级模式表。

实验步骤

- (1) 进入 SimpleScalar 目录 (simplesim-3.0)。
- (2) 用 sim-bpred 仿真器运行 SPEC2000 INT 中任意 3 个程序，分别采用 4 种不同的分支预测方法，即 bimod 方式, two-level adaptive 方式, always taken 方式, always not taken 方式，并对前两种分别使用下表中两种参数配置：分析仿真器输出的关于分支预测的统计参数集，填写表格，并对各仿真器的能力给出相应说明。

命令格式为：./sim-bpred {-option} executable_benchmark -argument

实验报告

包括在仿真器上运行的 3 个程序的结果统计数据表格，基于获得的 3 组数据，作图，并对各种分支预测方法的性能进行对比、分析。

程序 1

	Always taken	always not taken	bimod(512)	Bimod(1024)	Two level (1,1024,8 ,0)	Two level (1,64,6,1)
Prediction_rate						

程序 2

	Always taken	always not taken	bimod(512)	Bimod(1024)	Two level (1,1024,8 ,0)	Two level (1,64,6,1)
Prediction_rate						

程序 3

	Always taken	always not taken	bimod(512)	Bimod(1024)	Two level (1,1024,8 ,0)	Two level (1,64,6,1)
Prediction_rate						